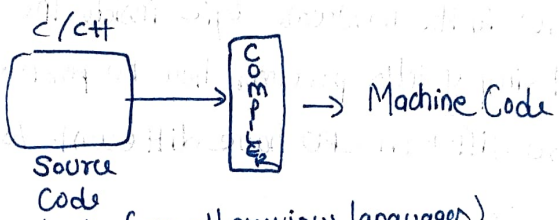
Lecture - 1 Introduction to Java

1980s / 1990's

C / C++ → These were the popular languages that existed before Java and these are some of the their features :-

1.) Fast
2.) Simple (as compared to other languages at that time)
3.) Low level → close to hardware

→ So why Java come, so one issue come with these languages that was they are platform dependent, so compiler compiled code can run on a specific platform computer only, to run on another platform, we need to again compile code that led to portability issue. So portability issue come with these languages or earlier languages.



C/C++ → platform dependent (or all previous languages)

platform → here compiled code by compiler will be different for different platforms. So binary code will be different for different platforms.

Processor	OS
eg→ Intel x86 +	Windows (P1)
or	
ARM +	Mac (P2)

* Why different binaries for different platforms.

eg→ suppose we have a code that print ("Hare Krishna")

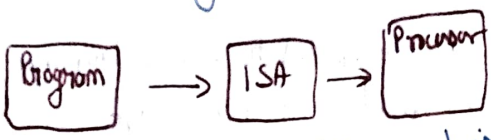
```
{
   "Hello Krishna
}
```

⇒ So in our source code we write lots of commands → in response, OS gives its system libraries code into compiled format so it states that these are methods calling will lead to a specific functionality, so its code gets includes into machine code. For different OS these methods to perform a specific functionalities are different, so their binary code will also be different. So one distinction is this that how due to different OS we see difference in compiled code.

eg→ print, file operations, Memory allocation etc.

⇒ Now here is a thing, different processors have different no of transistors and their arrangement is different depending upon the architecture of that CPU

So there will be difference in ways to interact for a specific compiled code for one CPU and other CPU. We will be needing different binaries to perform a functionality in different CPUs. So based on the architecture we will have different binary code to different CPU.



ISA (Instruction Set Architecture) is the formal specification of a processor's capabilities - the complete set of machine-level instructions (like add, load, jump), registers, data types, memory addressing modes, and the rules for how software talks to hardware, examples → x86, ARM and RISC-V. It is not stored in Bios or RAM as a file or document; rather it is built into processors design itself. The ISA is implemented in the hardware logic inside the CPU (control unit, ALU, decoders, etc). It simply tells processor how to perform commands like ADD, LOAD, JUMP, STORE. So different CPU have different ISA

⇒ Another reasons of why Java come?

2.) Simplicity → C/C++ was simpler than its previous languages, but still it had some functionalities / things that make a language complex, java simplified it and abstracted it.

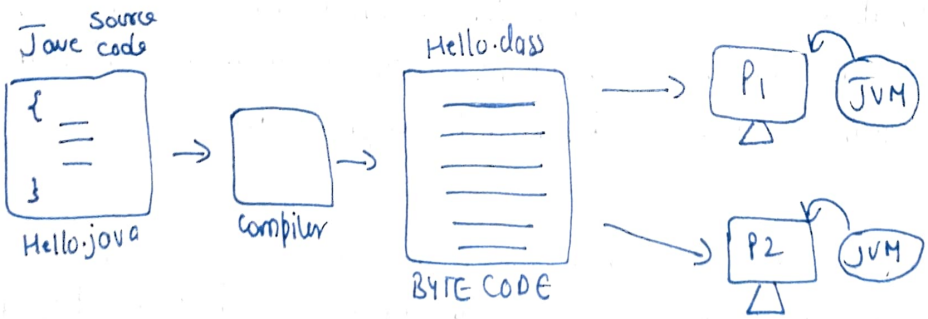
3.) Security → C/C++ was less secure, so there was a need of language that was more secure.

→ and due to all these 3 reasons JAVA come

1.) Portability
2.) Simplicity
3.) Security

* Solutions By Java

1.) Portability → Java introduced concept of Byte code to resolve this issue.



→ Hello.java file (source code) gets compiled by Java compiler into an intermediate file called ..class file (that is byte code). Now we have a software called JVM (Java Virtual Machine) that is platform specific that is it knows how to use OS system libraries methods and convert them to machine code. And also how to use ISA to convert byte code to machine code.

this is how JAVA resolves portability issues. So JVM converts byte code to machine specific machine code.

* Features of Java

1.) **Portable** → means compile java code to byte code and then platform specific JVM will convert it to machine code specific to that system. This is called WORA (write once Read Anywhere).

* Why Portability Became an issue

→ Portability become major issue in the early days of computing because different hardware architectures and os use different machine level instructions. When programs written in C/C++, they were compiled directly to platform specific machine code, meaning the compiled program could only run on the system for which it was built. If some application needed to run for another platform, it had to be recompiled for that environment, leading to increased development effort, maintenance complexity & higher cost.

→ deploying same code on servers with different platforms.

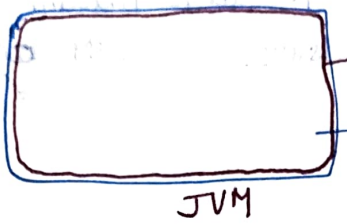
2.) **Simple** → C/C++ was simple as compared to other languages at that time, but it had some functionalities/features that used to make it complex for eg:-

(i) Pointers
(ii) Multiple Inheritance
(iii) Manual Memory allocation/deallocation

→ so Java removed it making things more simple.

3.) **Secure** → When java started to become popular, everyone started to use java to create backend → using servlet and also we could have made frontend using applets (that used to be transmitted from server to client browser and can run on user browser) using which frontend can be made. But with so many functionalities comes Security risk.

→ Java resolved this by making JVM secured as it will run bytecode in a secured environment inside JVM, so if that bytecode demands for any unauthorized access then JVM will deny it [Java Security]



→ JVM alone handles Portability and Security Issues. Now Java applets is discontinued.

* Can C/C++ be platform Independent

→ yes it can be made platform independent, but it served as a language that was very close to hardware and run fastly, its purpose was not to be platform independent or to serve everyone. But Microsoft did some experiments to make C/C++ platform independent but it didn't got successful. But they experimented it with C# and they made it platform independent using similar concept to JAVA.

Some languages that follow some principle of JAVA (in a slight diff. manner)

- C#
- python

→ but JAVA brought this platform Independent concept.